FLOCK ENERGY - DATA SCIENCE TAKE-HOME

REFIT data cleaning and washing-machine disaggregation

Raw House 1 cleaning · washing-machine EDA across 12 homes · seq2point-style XGBoost validated leave-one-house-out

Code: https://github.com/ArpitThakur7/flock-energy-refit

Contents

1. Summary and recommendation
2. Overview, setup and design
3. Data, quality and cleaning
4. EDA, model results and failure analysis

Recommendation: washing-machine disaggregation for utility use

Bottom line: using only a home's 1-minute whole-house meter, the model finds about **8 in 10 washes** in homes it has never seen. Its **daily** washing-machine energy estimate is 28% more accurate than assuming zero. It is ready for **aggregated and pattern-based insights** (when and how often a home washes, which homes are heavy users). It is **not** ready for exact per-home kWh figures on a bill. Tested on 12 UK homes from REFIT, 2013–2015, holding out one home at a time.

Suitable uses

Use	Why it is supported
Detect whether, how often and when a home washes	81% of washes found; the start-hour pattern is reliable
Rank or segment customers (heavy washers, peak-time washers)	Wash days are clearly separated from non-wash days in most homes
Portfolio-level estimates (e.g. washing load at 08:00–11:00 across a feeder)	Errors cancel out across homes: −3% total energy error pooled over 12 homes

Not suitable (yet)

- **An exact kWh figure for one home's washing machine.** Single-home totals can be off by ±50–100%.
- **Automated decisions or penalties for an individual customer.** About 6 in 10 predicted washes are other heating appliances such as showers, ovens and dishwashers.
- **Homes with unusual machines.** One test home with a short-burst heater was mostly missed. **Data at 15 or 30 minutes**: see below.

Section 4 answers

Opportunity: shift washes out of the morning peak and toward cheap off-peak hours

Finding. 44% of washes start between 07:00 and 12:00, and the busiest slot is Saturday 09:00. Two of the 12 homes already use delay timers to wash at night: House 19 starts 58% of washes between 22:00 and 01:00, and House 1 starts 21% between 03:00 and 06:00. Each wash draws about **2 kW for 10–25 minutes**, a large, short and **movable** block. Nearly every modern machine has a delay-start button, so shifting a wash needs no new hardware.

Offer. Customers the model flags as frequent morning or evening-peak washers get a tailored message ("Your washing machine usually runs around 9 am. Setting the delay timer to finish by 7 am on the off-peak rate would save about £X a year"). Where a time-of-use tariff or a demand-flexibility event exists, link it to that.

Measuring whether it worked.

- *Design:* a randomised controlled trial. Randomly split the flagged customers into a group that gets the message and a control group that does not, and record a baseline period of 4–8 weeks before sending.
- *Primary metric:* the share of detected washes starting in peak hours (e.g. 07:00–11:00 and 16:00–19:00), per customer, before vs after, treatment vs control (difference-in-differences). The same detector is applied to both groups, so its errors affect them equally and the *difference* stays valid.
- *Secondary metrics:* peak-hour kWh and the 09:00 load at feeder level, and bill savings for customers on time-of-use tariffs.
- *Ground truth check:* a short in-app or SMS question ("Did you use the delay timer this week?") for a sample of customers, which also gives labelled data to improve the model.

Total consumption barely changes, because a wash uses the same energy whenever it runs. The value is in **peak reduction and lower cost for the customer**, and the evaluation should be framed that way.

Limitations

- **Overlapping appliance signatures.** The washing machine's main energy is a ~2 kW heating plateau, and kettles, showers, ovens, dishwashers and tumble dryers create similar blocks. This is the largest source of error (about 61% of predicted washes are false) and the reason we recommend aggregated and pattern-based outputs.

- **Missing data.** Homes lose 6–25% of minutes to outages, including gaps of weeks. We never fill long gaps. Any customer-facing figure must be calculated over available days and say so (e.g. "based on 26 of 30 days"). The official REFIT "Issues" flag would remove a third of washing-machine minutes, so we did not rely on it.
- **Household differences.** Washes per week range from 2 to 11, and energy per wash from 0.24 to 0.72 kWh, by machine and programme. A machine unlike those in training (House 19) is mostly missed. Performance in a new market must be checked on a labelled pilot before launch.

- **Applying UK results to India.** The UK homes are mostly detached houses with heater-equipped washers that heat their own water. Indian homes differ in several ways that matter:

    - Many machines (semi-automatic top-loaders, many front-loaders on cold washes) have **no heater**, so the 2 kW plateau, the model's strongest clue, is absent. A wash becomes a 200–500 W motor pattern, much harder to separate.
    - Other dominant loads differ: air conditioners, water heaters (geysers), water pumps, and inverter/UPS battery charging all create large blocks.
    - There is more voltage variation and more power cuts.
    - Metering is often single-phase and at lower resolution.

    The pipeline (cleaning, cycle rule, leave-one-house-out validation) transfers, but **the model must be retrained and re-validated on labelled Indian homes**, and the cycle thresholds re-set (e.g. a lower peak threshold for machines without heaters).

If only 15- or 30-minute smart-meter data were available

At 15–30 minutes a single reading averages over most or all of a wash's 10–25 minute heating plateau. The 2 kW block becomes a modest bump of a few hundred watts, the same size as a kettle used a few times or a cooker on low. The tail, start and end that make a wash recognisable are averaged away, and a wash that starts mid-interval is split across two readings. Telling washes apart from other loads would become much harder: we would expect reliable detection only for hot, long programmes in otherwise quiet periods, and much more confusion during busy morning and evening hours. Estimates would have to move to **daily or weekly totals and probabilistic statements** ("likely 3–5 washes this week"). The model would also lean more on habits (time of day, day of week) and household information, so it would generalise worse to unusual homes. Timing insights such as "you wash at 9 am" would still be possible to the nearest half-hour. Exact cycle counts and per-wash energy would not.

REFIT data cleaning and washing-machine disaggregation

A Flock Energy take-home on the REFIT Electrical Load Measurements dataset (20 UK homes, 2013–2015). It is a small, reproducible pipeline that:

1. **Cleans raw House 1 data** to 1-minute resolution, fixing a daylight-saving error in the raw timestamps, merging overlapping files, removing sensor error codes and duplicates, and flagging outages. The result matches the official cleaned data with correlation 0.997.
2. **Analyses washing-machine use** in 12 homes from the official cleaned data: 4,566 detected cycles, usage by hour and weekday, and cycle duration, energy and peak compared across homes.
3. **Estimates washing-machine power from the whole-house meter** with a seq2point-style XGBoost model. It is validated leave-one-house-out against two baselines. On homes it has never seen, it finds 81% of washes and cuts daily energy error by 28% compared with predicting zero.

Document	Contents
DATA.md	Raw data coverage and quality findings, cleaning steps, gap and outlier decisions, remaining concerns
RESULTS.md	Washing-machine EDA, plots, model results, validation, failure analysis, cited papers
Recommendation.md	One-page utility summary and the Section 4 answers

Setup and run

Requires Python 3.10+ (tested on 3.13, Windows 11) and roughly 12 GB of free disk space for the extracted csvs.

```
pip install -r requirements.txt
python run_all.py
```

`run_all.py` runs these scripts in order (each can also be run alone from `src/`):

Step	Script	Output
Data	`src/get_data.py`	Downloads the two `.7z` archives (~1.2 GB) and extracts only the files used into `data/raw` and `data/clean`. If you already have the `CLEAN_House*.csv` files, put them in `data/clean/` and they are reused.
Part 1	`src/part1_clean_raw.py`	`data/interim/raw_h1_cleaned_1min.parquet`, `outputs/raw_h1_*.csv`, `figures/p1_*.png`
Part 2	`src/part2_wm_eda.py`	`outputs/wm_quality.csv`, `wm_cycles.csv`, `wm_cycle_summary.csv`, `figures/p2_*.png`
Part 3	`src/part3_model.py`	`outputs/model_metrics*.csv`, `model_daily_energy.csv`, `model_feature_importance.csv`, `figures/p3_*.png`
Ablation	`REFIT_NO_TIME=1 python src/part3_model.py`	`outputs/*_notime.csv` (model without time-of-day features)
Report	`python tools/build_report_pdf.py --repo-url <url>`	`Flock_Energy_REFIT_submission.pdf`: the four docs plus figures in one PDF under 1 MB (needs Edge or Chrome)

The full run takes about 5 minutes on a laptop. XGBoost uses the GPU if one is available and falls back to the CPU otherwise. Seeds are fixed (`SEED = 42`). GPU training can still vary in the last decimal place.

> The download links in the brief (`pureportal.strath.ac.uk/files/...`) return HTTP 403. `get_data.py` uses the equivalent `pure.strath.ac.uk/ws/portalfiles/portal/...` links.

Repository layout

```
src/config.py          every path, house selection and threshold in one place
src/clean_data.py      load an official cleaned csv -> 1-minute parquet cache
src/cycles.py          washing-machine cycle rule (used for EDA and model filtering)
src/part1_clean_raw.py, part2_wm_eda.py, part3_model.py, get_data.py
```

```
run_all.py            end-to-end runner
figures/  outputs/    generated plots and tables (committed so results are viewable)
data/                 not committed (downloaded by get_data.py)
```

Key assumptions and preprocessing decisions

- **Clock.** The `Unix` column is the clock. Raw Part1 is converted from UK local time to UTC (DATA.md explains the evidence). All resampling is done in UTC to avoid daylight-saving duplicates, and local time is used only for hour-of-day analysis.
- **1-minute resampling** uses the **mean** of the samples in each minute, so energy totals are kept. A minute with no samples stays **NaN**.
- **Gaps:** gaps of 5 minutes or less are forward-filled. Longer gaps are flagged and never filled.
- **Invalid values:** IAM > 4 kW, aggregate > 15 kW, negative values, and any appliance reading above the whole-house reading. These become **NaN, not 0**, so they are not mistaken for "off".
- **Duplicates:** exact duplicate rows are dropped. Rows that share a timestamp but differ are merged, keeping the last non-missing value per column.
- **Washing-machine target:** the readme's washing-machine channel. House 4 has two washing machines, so its target is their sum.
- **The REFIT `Issues` flag is not used to drop data.** It fires on 17–47% of the minutes when the washing machine is running, so dropping those minutes would remove the events being modelled. Only minutes where the washing machine itself exceeds the aggregate are excluded.
- **Cycle rule:** above 20 W counts as running; pauses of 15 minutes or less are bridged; a cycle needs at least 10 minutes, 0.05 kWh and a 200 W peak.

Validation design

Leave-one-house-out over 12 homes. Each home is the test home once and is never used for training, early stopping or threshold selection in that fold. Inside the training homes, the last 20% of each home's timeline (by time) is used for early stopping. Overlapping 61-minute windows therefore cannot leak between train and test, and the test mimics the real use case: a new customer with no sub-metering. The baselines are always-off and an hour-of-week usage profile.

Trade-offs

- **XGBoost on window features instead of a CNN (Seq2Point).** It is much faster to train 12 folds, easier to explain and inspect (feature importance, ablation), and good enough to clearly beat the baselines. The cost is that hand-made features probably capture the low motor tail less well than a CNN would.
- **Imbalance handling.** Every minute near a cycle is kept, plus a 10% sample of other minutes with a weight of 10. This keeps training at about 1.1 M rows per fold without biasing the loss.
- **Cycle filter after prediction.** It removes short false alarms and makes total energy much less biased, at the cost of some recall on unusual machines (House 19).
- **Metrics.** Headline metrics are daily-energy and cycle-level, because per-minute MAE rewards predicting zero for an appliance that is off more than 90% of the time. MAE is still reported.
- **House choice.** 12 of 20 homes are used. Homes with an ambiguous or changing washing-machine channel (6, 8, 9, 10, 11, 12, 13, 20) were left out rather than guessed at.

AI tools used

I wrote this repository with the help of **Claude (Anthropic) in Claude Code**, used as a pair programmer. It helped to:

- find a working download path for the data;
- write and debug the pipeline code;
- investigate data issues: it ran the checks that revealed the Part1 local-time offset, the power-of-two glitch values, and the bias in the `Issues` flag;
- draft the markdown documents.

All decisions, numbers and plots were checked against the pipeline's actual outputs. Every figure and table in the docs is produced by the code in this repo.

Possible improvements

1. A second-stage, event-level classifier that confirms predicted cycles, to reduce false alarms (currently ~61% of predicted washes).
2. A Seq2Point CNN under the same leave-one-house-out folds, for comparison.
3. More homes after manual channel checks, plus per-home adaptation from a few labelled washes or user feedback.
4. Prediction intervals for monthly washing-machine kWh, so customer-facing insights can be held back when uncertain.
5. Unit tests for the cycle rule and the timezone conversion. These are checked indirectly today, through the official-data comparison and the plots.

Data licence and citation

REFIT data © University of Strathclyde, CC-BY 4.0. Murray, D., Stankovic, L. & Stankovic, V. *An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study.* Scientific Data 4, 160122 (2017).

DATA.md: coverage, quality findings and cleaning decisions

1. Dataset

REFIT Electrical Load Measurements (University of Strathclyde; Murray et al., 2017, *Scientific Data* 4:160122). CC-BY 4.0.

Households	20 homes near Loughborough, UK (numbered 1–21, there is no House 14)
Period	Sep 2013 – Jul 2015 (each house covers roughly 15–21 months)
Channels per house	1 whole-house current clamp ("Aggregate") + 9 Individual Appliance Monitors (IAMs, plug meters)
Quantity / unit	Active power, **watts**
Native sampling	Polled every ~6–8 s. Sensors are **not synchronised**: each reading may be up to 8 s old.
Analysis resolution	**1 minute**, mean of the samples in each UTC minute (keeps energy totals)

Files used

Part	Files	Source archive
1 (raw cleaning)	`RAW_House1_Part1.csv`, `RAW_House1_Part2.csv`	`REFIT_RAW_081116.7z`
2, 3 (EDA, model)	`CLEAN_House{1,2,3,4,5,7,15,16,17,18,19,21}.csv`	`CLEAN_REFIT_081116.7z`

> **Download note:** the `pureportal.strath.ac.uk/files/...` links in the brief return HTTP 403. The same files are served at `pure.strath.ac.uk/ws/portalfiles/portal/<id>/<name>`, and `src/get_data.py` uses those links.

Known dataset limitations (from the readme and our checks)

- Appliance labels describe what was plugged in *at installation*. Householders may have moved IAMs later, and the readme lists several changes.
- "Television site" and "computer site" IAMs can include several devices.
- IAMs only measure plug loads. Hard-wired loads (electric shower, oven, immersion heater) appear only in the aggregate, so they are never directly labelled.
- The cleaned files forward-fill short NaNs and replace IAM spikes above 4 kW with 0. Long outages are simply absent (no rows).
- There are periods of missing data, notably February 2014. It was caused by internet, hardware and routing failures.

---

2. Part 1: Raw House 1 inspection findings

Produced by `src/part1_clean_raw.py`, which writes `outputs/raw_h1_inspection.csv`.

Check	Part1 (Oct 2013 – Oct 2014)	Part2 (Jun 2014 – Jul 2015)
Rows	4,377,877	4,140,886
Timestamp parse failures	0	0
Out-of-order steps	**1** (27 Oct 2013 clock change, see below)	0
Duplicate timestamps	**1,004,404**	14,755
… of which exact duplicate rows	884,307	393
Median / 99th pct. interval	3 s / 15 s	7 s / 9 s
Gaps > 5 min / > 1 h	380 / 14	72 / 19
Longest gap	**998 h (41.6 days, 24 Jan – 6 Mar 2014)**	208 h
Missing cells	0% (unavailable sensors were written as **0**)	12.8% (written as **NaN**)
Aggregate > 15 kW	82 (max 24,000 W)	104 (max 29,159 W)
IAM > 4 kW	3,038	911
Largest IAM value	98,301 W	98,301 W
Negative values	0	0

Key findings

1. **The Part1 timestamps are in local time, not UTC.** Part1 was logged in UK wall-clock time but stored as if it were UTC. The evidence:

    - The only backwards step is `2013-10-27 01:59:59 → 01:00:01`, which is the autumn clock change: the hour 01:00–02:00 appears twice.
    - The 01:00 hour is missing on 30 Mar 2014 (the spring clock change).
    - When raw readings are matched to the official cleaned file, Part1 lines up at an offset of **−3600 s from April to October** and **0 s from November to March**. Part2 lines up at 0 s all year, so Part2 is already UTC.

    Ignoring this would misplace every summer reading by an hour, and it would put one autumn hour in the wrong order.

2. **The two files overlap from 27 Jun to 1 Oct 2014.** In that window they disagree: only 20% of shared timestamps carry the same aggregate value. The official cleaned file follows Part1 there.

3. **Sensor error codes.** IAM values of 4096, 8192, 16384, 32767, 65535 and 98301 W are integer-overflow or bit-error codes. A plug socket cannot deliver more than about 3 kW. Glitches also occur at lower values: for example, the washing-machine IAM reads 2048 W while the whole house is drawing 190 W.

4. **Massive duplication in Part1.** A quarter of Part1 rows repeat an earlier timestamp, and most of them are exact copies. Part2 instead spreads one poll over several partial rows, each holding a different subset of sensors with NaNs in the others.

5. **Aggregate above 15 kW.** There are 186 readings in total (up to 29 kW). They are not physically impossible for a 100 A supply, but they far exceed this home's normal peak of about 10–12 kW, which comes from short, kettle- or shower-like spikes. I treat them as invalid.

---

3. Part 1: Cleaning pipeline (House 1 raw → 1 minute)

Each step and its count is logged to `outputs/raw_h1_steps.csv`.

#	Step	Rule	Effect
1	Parse	`Unix` column is the clock; `Time` is only its string form	0 failures
2	Timezone	Part1: treat Unix as `Europe/London` wall time and convert to UTC. In the repeated hour the **file order** decides BST (first pass) vs GMT (second pass). Part2 is unchanged.	2,795,917 Part1 rows shifted 1 h
3	Overlap	Use Part1 up to its last reading (1 Oct 2014 00:27 UTC) and Part2 after that. Part1 is denser in the overlap and matches the official data.	944,950 Part2 rows dropped
4	Invalid values → NaN	IAM > 4,000 W, aggregate > 15,000 W, negative values. **NaN, not 0**, so they are not mistaken for "off".	3,895 IAM + 171 aggregate readings
5	Sort and duplicates	Stable sort by timestamp. Drop exact duplicate rows. Rows with the same timestamp but different values are merged per column, **keeping the last non-missing value** (the most recent poll). This also re-assembles Part2's split rows.	884,254 dropped, 131,831 merged
6	Resample	Mean per UTC minute	920,091 minutes
6b	Consistency	An IAM minute above 1.1 × aggregate + 50 W is impossible (one appliance can't use more than the whole house) → NaN. This catches glitches under 4 kW.	3,821 IAM-minutes (2,498 on the electric heater)
7	Short gaps	Gaps of **≤ 5 min** are forward-filled. A sensor that skips a report usually means "no change", and 5 min is short compared with any appliance cycle.	812 minutes filled
7	Long gaps	Gaps **> 5 min are left as NaN**, flagged in an `outage` column and listed in `outputs/raw_h1_outages.csv`	408 outages, 121,355 min (13.2% of the span)

Outage profile: 384 outages of 5–60 min, 12 of 1–24 h, and 12 longer than a day. The longest is 41.6 days (24 Jan – 6 Mar 2014), followed by 8.6 days (May 2015) and 7.5 days (June 2015). These are not filled because there is no defensible way to invent days of appliance behaviour. They stay NaN, and downstream steps must skip them.

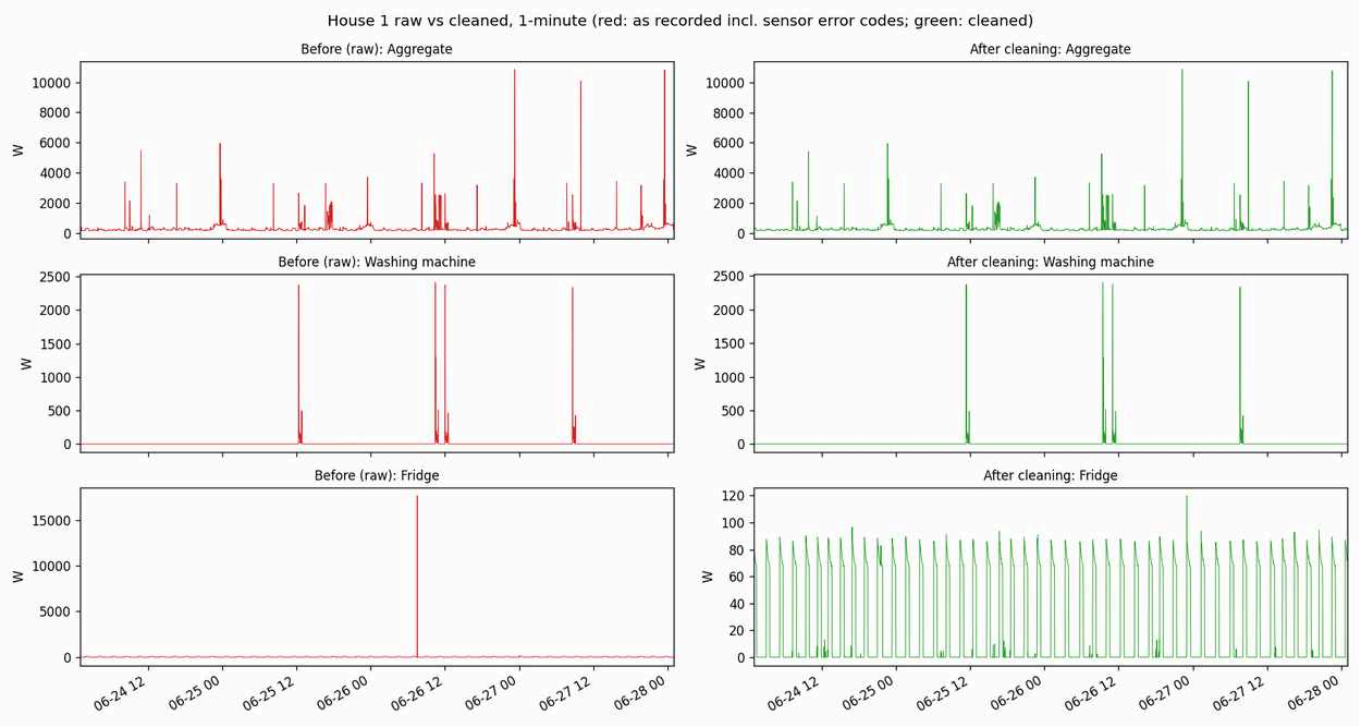
Validation against the official cleaned file (1-minute aggregate, 797,815 shared minutes):

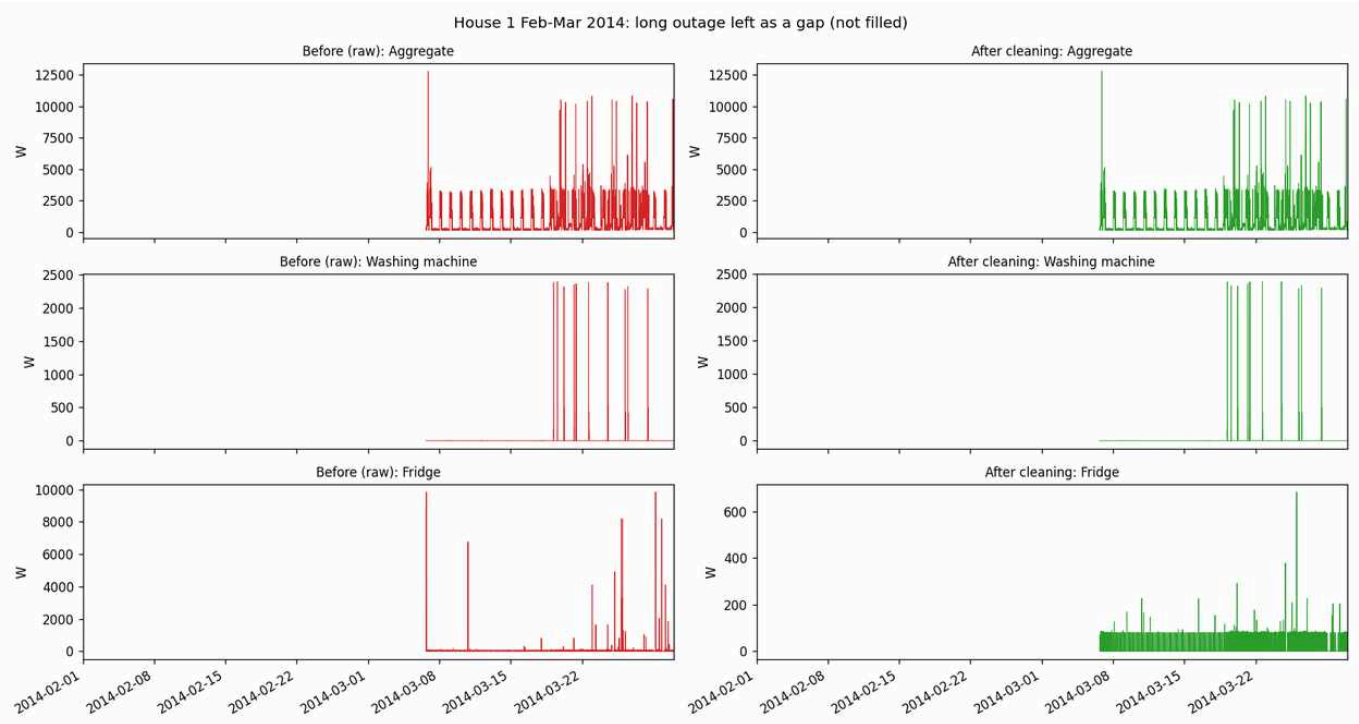
	Correlation with official	Mean absolute error
Raw as recorded (no cleaning)	0.768	89.5 W
Our pipeline	**0.997**	**4.4 W**

So the simple pipeline reproduces the official cleaning very closely. Nearly all of the improvement comes from the timezone fix and the overlap rule.

Figures

- `figures/p1_before_after_week.png`: aggregate, washing machine and fridge for 4 days, before and after cleaning. Sensor-code spikes (fridge "2,000 W", washing machine "4,096 W") disappear, and the summer data moves by 1 h.
- `figures/p1_before_after_outage.png`: Feb–Mar 2014. The 41-day outage is kept as a visible gap.
- `figures/p1_daily_coverage.png`: the percentage of minutes with a reading, day by day.





Two differences from the official pipeline. We set invalid IAM values to NaN, where the official file sets them to 0. We also do not re-map IAMs that householders moved between appliances. That would need per-house manual review, which the brief puts out of scope.

---

4. Parts 2–3: Cleaned data used for the washing-machine analysis

Houses and channels. I used every house where the readme names a washing machine and there is no ambiguity (12 houses):

House	WM channel(s)	Notes
1	Appliance5	House also has a **washer-dryer** (Appliance4), which is not in the target: a confounder
2, 3, 7, 15, 16, 17, 18, 19, 21	as listed in `src/config.py`	
4	Appliance4 + Appliance5	Two washing machines. The target is their **sum**, because both leave the same footprint in the aggregate.
5	Appliance3	The model (AEG L99695HWD) is actually a **washer-dryer**, so its drying cycles look different

Houses 8 and 9 are excluded because their "washing machine" and "washer dryer" channels are hard to tell apart. Houses 6, 10, 11, 12, 13 and 20 are excluded because the channel is missing, unlabelled (`???`) or changes part-way through. They could be added later.

Resampling. Mean per UTC minute of the ~8 s samples (`src/clean_data.py`). A minute with no samples stays NaN, so outages are counted, not filled. Local time (`Europe/London`) is only used for hour-of-day and weekday analysis.

Availability and quality (written to `outputs/wm_quality.csv`)

House	Period	Missing minutes	Gaps > 1 h	Issues flag (all min.)	WM ≤ 20 W	Cycles	Cycles/week
1	2013-10-09 → 2015-07-10	12.6%	22	2.1%	98.1%	379	4.8
2	2013-09-17 → 2015-05-28	24.6%	78	2.0%	95.3%	320	4.8
3	2013-09-25 → 2015-06-02	11.4%	17	7.7%	94.3%	633	8.1
4	2013-10-11 → 2015-07-07	13.7%	212	3.1%	98.5%	256	3.3
5	2013-09-26 → 2015-07-06	9.3%	203	8.4%	92.3%	693	8.3
7	2013-11-01 → 2015-07-08	12.6%	23	4.6%	91.6%	834	10.9
15	2013-12-17 → 2015-07-08	13.7%	198	1.6%	97.1%	249	3.6
16	2014-01-10 → 2015-07-08	17.3%	201	1.0%	96.3%	307	4.8
17	2014-03-06 → 2015-06-19	6.2%	13	4.3%	97.6%	279	4.4
18	2014-03-07 → 2015-05-24	5.8%	14	5.8%	98.8%	132	2.2
19	2014-03-06 → 2015-06-20	6.5%	12	4.1%	96.1%	255	4.1
21	2014-03-07 → 2015-07-10	9.4%	15	5.4%	96.9%	229	3.6

The washing machine is "off" (≤ 20 W) for 92–99% of available minutes. That class imbalance drives the metric choices in RESULTS.md.

The official `Issues` flag is biased toward washing-machine use

The readme suggests discarding rows where `Issues = 1` (sum of the 9 IAMs > aggregate). We checked when it fires:

House	1	2	3	4	5	7	15	16	17	18	19	21
Issues share of all minutes	2%	2%	8%	3%	8%	5%	2%	1%	4%	6%	4%	5%
Issues share of **WM-on** minutes	33%	17%	27%	41%	30%	23%	36%	5%	47%	22%	34%	37%
WM alone > aggregate (WM-on minutes)	5.0%	0.4%	4.5%	2.8%	4.9%	0.9%	1.7%	0.2%	2.6%	0.4%	1.2%	9.0%

The flag fires **5–10× more often while the washing machine runs**. When nine unsynchronised meters are summed during a 2 kW load, a few seconds of lag easily pushes the sum above the clamp reading. Dropping flagged minutes would

therefore remove a large part of the very events we want to model. **Decision:** keep flagged minutes, and exclude only minutes where the *washing machine on its own* exceeds 1.1 × aggregate + 50 W (physically impossible for the target).

---

5. Remaining concerns

- **Appliance re-assignment.** I trust the readme's channel labels. If an IAM was moved to another appliance part-way through, the "ground truth" is wrong for that period.
- **Back-to-back washes.** The 15-minute bridging rule merges washes run one after another into one long cycle (see House 5 and House 21 in `p2_detected_cycles.png`). This inflates per-cycle duration and energy and lowers cycle counts somewhat.
- **Washer-dryers** (House 5, and the confounding one in House 1) have drying phases that do not look like washing.
- **Unlabelled big loads.** Electric showers, ovens and immersion heaters are in the aggregate but never metered. They are the main source of false positives (see RESULTS.md).
- **Part1 zeros.** Before Part2, unavailable sensors were written as 0, so in Part1 "off" and "not reporting" cannot be told apart. This mainly affects IAM channels, because the aggregate is almost never 0.

RESULTS.md: washing-machine EDA, model results and failure analysis

The data is the official cleaned REFIT data at 1-minute resolution. It covers **12 houses** (1, 2, 3, 4, 5, 7, 15, 16, 17, 18, 19 and 21) over each house's full recorded period, Sep 2013 – Jul 2015 (per-house dates are in DATA.md §4). The houses yield **4,566 detected washing-machine cycles**. Every number below is regenerated by `python run_all.py`.

Part 2: Exploratory analysis of washing-machine use

Cycle rule (plain language)

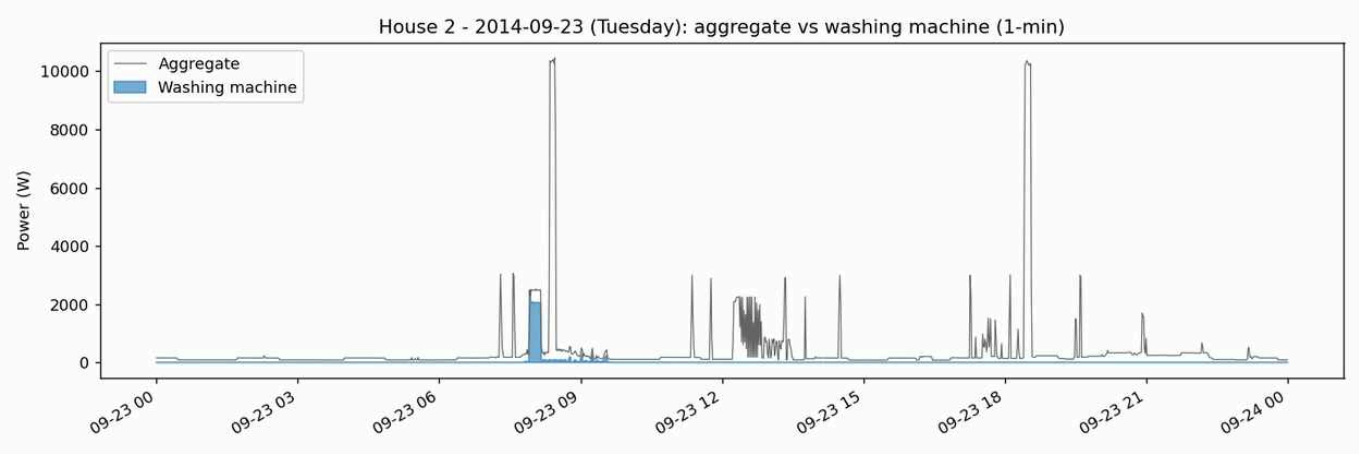
> A minute counts as **running** when the washing machine's average power is **above 20 W**, which is well above standby (0–5 W). A wash has quiet phases (soaking, draining, pauses between tumbles), so running minutes separated by a pause of **15 minutes or less** are joined into one cycle. A joined block counts as a **cycle** only if it lasts **at least 10 minutes**, uses **at least 0.05 kWh** and reaches **at least 200 W** at some point. This removes blips such as the door lock or the display waking up.

The 20 W on-threshold matches what NILM work on washing machines commonly uses (e.g. Kelly & Knottenbelt, 2015). The rule is in `src/cycles.py`, and every threshold is in `src/config.py`.

Availability and data quality (summary; full table in DATA.md §4)

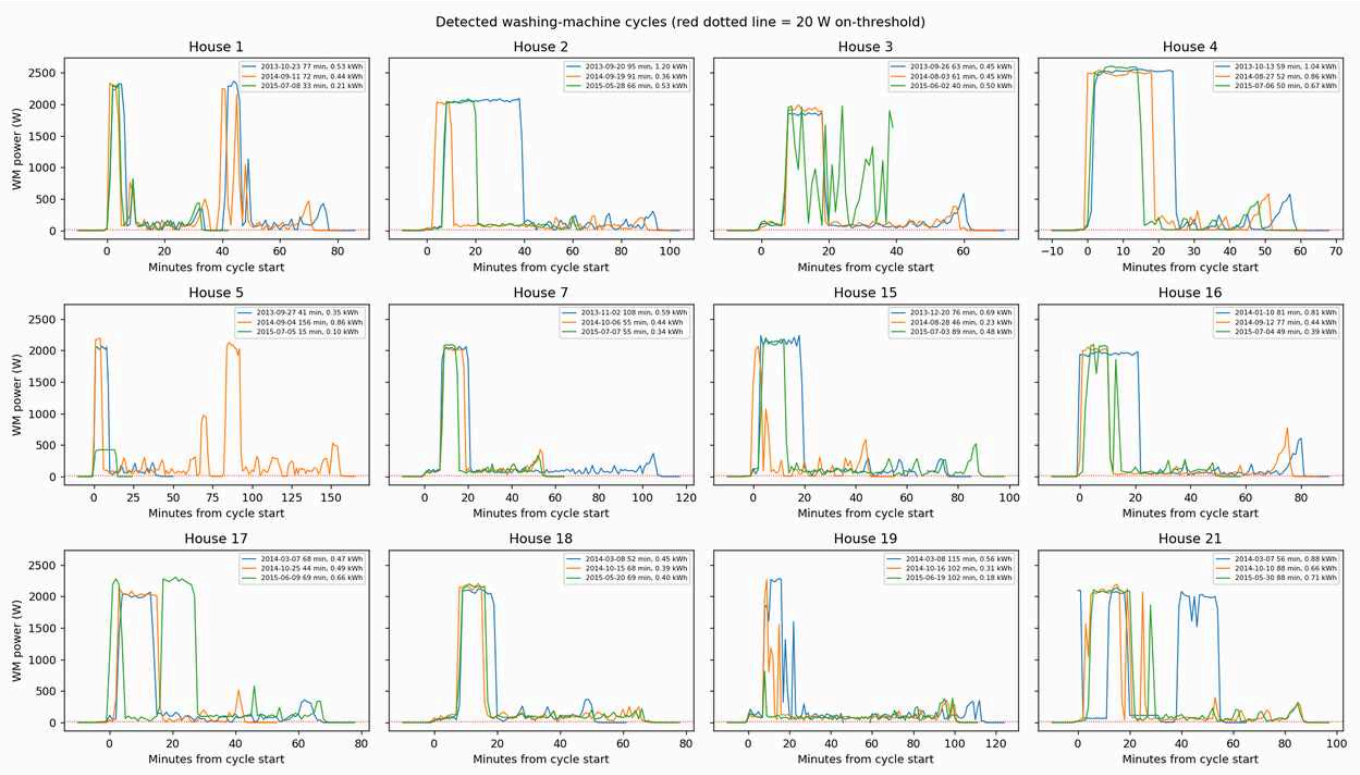
- Each house has 15–21 months of data. Between 6% and 25% of minutes are missing (House 2 is the worst, with 78 gaps over 1 h and one 62-day outage).
- The washing machine is off (≤ 20 W) for **92–99%** of available minutes and draws exactly 0 W for 52–99%. House 5's machine idles at a few watts rather than 0.
- Washes per week range from **2.2 (House 18)** to **10.9 (House 7)**, and washing-machine energy from 0.12 to 0.91 kWh/day, which is 1–7% of household consumption.

(i) A representative 24-hour trace: `figures/p2_representative_day.png`



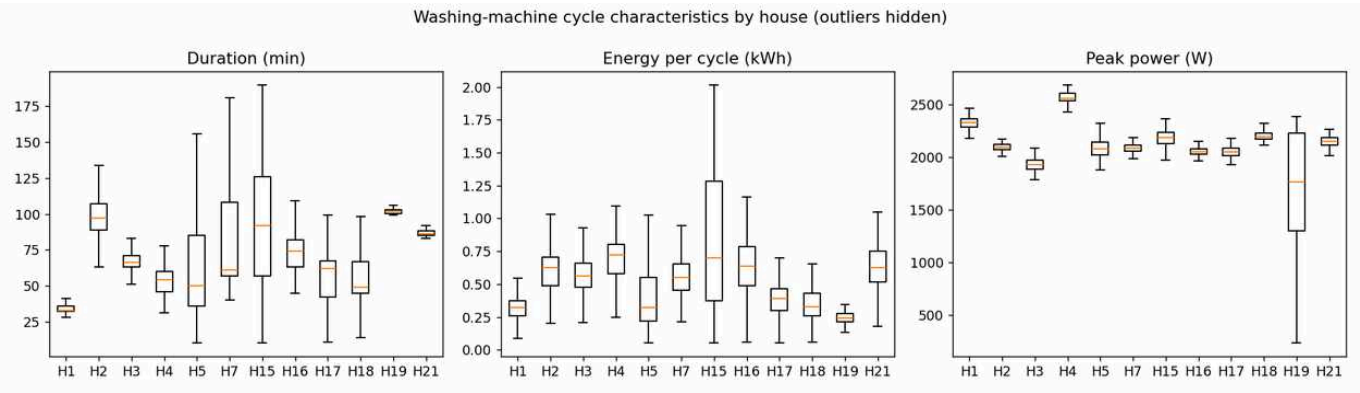
House 2 on Tuesday 23 Sep 2014, a day with complete data and a cycle of typical energy for that house. The wash starts around 07:50 with a **~2 kW heating plateau lasting about 10 minutes**. That is the element heating the water, and it accounts for most of the cycle's energy. A long low tail of 50–300 W follows (drum motor, rinses, spin). The aggregate also shows other large loads the model must not confuse with the wash: two ~10 kW spikes (most likely an electric shower) and repeated ~3 kW kettle-like spikes. **Interpretation:** washing-machine energy is dominated by one heating block, and that block looks like many other heating appliances. The tail is the distinctive part, but it is small compared with background noise.

(ii) Detected cycles: `figures/p2_detected_cycles.png`



The figure shows three cycles per house (earliest, middle and latest), aligned at cycle start. Almost every house shows the same two-phase shape: **a 1.8–2.6 kW plateau of 5–25 min, then 30–80 min of low motor activity**. Some cycles show two plateaus. These are hotter or longer programmes, or two washes run back to back and merged by the 15-minute rule (House 5 2014-09-04, House 21 2014-03-07). House 19's machine is different: it heats in short bursts rather than one plateau, which matters for the model (see failure cases).

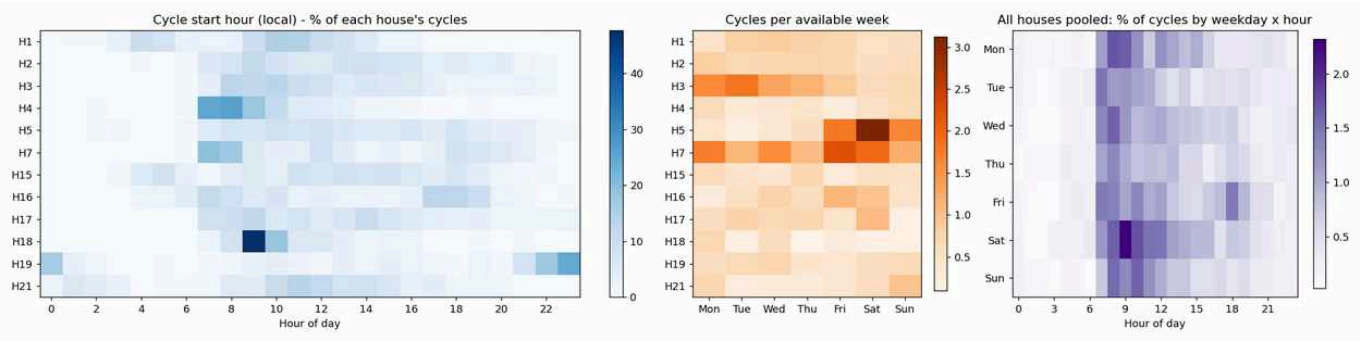
(iii) Cross-house comparison: `figures/p2_cross_house.png`



House	Cycles	Median duration (min)	Median energy (kWh)	Median peak (W)	Weekend share	Most common start hour
1	379	33	0.32	2,327	23%	10:00
2	320	97	0.63	2,091	24%	09:00
3	633	66	0.56	1,929	16%	10:00
4	256	54	0.72	2,556	36%	08:00
5	693	50	0.32	2,076	58%	10:00
7	834	61	0.55	2,083	29%	07:00
15	249	92	0.70	2,181	29%	11:00
16	307	74	0.64	2,050	30%	17:00
17	279	62	0.39	2,046	28%	09:00
18	132	49	0.33	2,191	15%	09:00
19	255	102	0.24	1,759	29%	23:00
21	229	86	0.63	2,145	43%	11:00

Interpretation: peak power is very consistent (about 2 kW, set by the heating element). Duration and energy per cycle differ by a factor of three between homes. House 1's machine runs short, low-energy programmes (about 33 min, 0.32 kWh), while House 4 averages 0.72 kWh per wash. Programme choice (temperature) matters as much as how often people wash. A home with few but hot washes can use more than a home with many cool ones.

Usage patterns by hour and weekday: `figures/p2_usage_patterns.png`



- **Hour of day:** pooled over all houses, **44% of washes start between 07:00 and 12:00**, peaking at 08:00–10:00, with a smaller evening shoulder (17:00–19:00). There are two clear exceptions. **House 19 starts 58% of its washes between 22:00 and 01:00**, and **House 1 starts 21% of its washes between 03:00 and 06:00**. Both look like delay-timer use, most likely to catch a cheaper night tariff (Economy 7).
- **Day of week:** most homes are flat across the week, but some differ strongly. House 5 washes mainly Friday to Sunday (58% at weekends), House 3 and House 7 wash most at the start of the week, and the pooled peak is **Saturday 09:00**.

Part 3: Disaggregation model

Task, homes and periods

Estimate the washing machine's power each minute from the **household aggregate only**, at 1-minute resolution. All 12 houses above are used over their full periods: 8.4 M usable minutes in total, after dropping minutes with a missing window or a physically impossible target (DATA.md §4).

Model: a seq2point-style gradient-boosted regressor (`src/part3_model.py`)

- **Idea from the literature:** sequence-to-point learning (Zhang et al., 2018). Predict the appliance's power at the *midpoint* of a window of aggregate readings, so the model can see both how a cycle starts and how it ends. I kept the windowing idea but replaced the CNN with **XGBoost** on hand-made window features. That is faster to train, simpler to explain, and model complexity is not assessed.
- **Features** (all from the aggregate in a ±30 min window): 31 lags; level above the local 2-hour minimum, which removes each house's different always-on load; rolling mean, std, min and max over 5–61 min; the fraction of time above 1.5 kW; first difference and its rolling absolute sum; plus hour of day and weekend. The time features are tested by an ablation below.
- **Training sample:** every minute within 60 min of a real cycle, plus a random 10% of all other minutes. Those other minutes get a **weight of 10**, so the loss still reflects the true balance between on and off minutes.
- **Post-processing:** predictions are clipped to `[0, aggregate]`, values below 20 W become 0, and a **cycle filter** keeps predicted power only inside predicted blocks that pass the same cycle rule as Part 2. This removes short false alarms.

Validation design: leave-one-house-out (12 folds)

Each house is the **test home exactly once**. In that fold it contributes nothing to training, early stopping or threshold choice. Within the 11 training houses, the **last 20% of each house's timeline** is held out for early stopping and for choosing the zeroing threshold.

How this avoids leakage: neighbouring 1-minute windows overlap by 60 of 61 minutes. A random split would put almost identical windows in both train and test and grossly overstate accuracy. Splitting by **household** guarantees the test home's appliances, habits and wiring are never seen, which is also the real deployment setting (a new customer with no sub-metering). The inner split is by **time**, so early stopping cannot peek at future data from the same home either.

Baselines

- **Always-off:** predict 0 W. This is hard to beat on plain per-minute error, because the machine is off more than 90% of the time.
- **Hour-of-week profile:** for each weekday and hour, the average washing-machine power across the training houses. It knows typical habits but not the test home's aggregate.

Metrics and why

Metric	What it tells a utility
Daily energy MAE (kWh/day)	How wrong the estimated washing-machine energy per day is. This is the closest to what a bill insight needs.
Total energy error (%)	Bias over the whole period, i.e. whether the model systematically over- or under-estimates.
Cycle recall / precision	Whether we catch each wash, and how many "washes" we report are real. This is what event-based insights (counting washes, timing) need.
Minute F1 (on = > 20 W)	Whether the on/off state is right minute by minute
MAE (W), and MAE during on-minutes	Standard NILM point error. It is dominated by the off-time and favours always-off, so it is reported but not used alone.

Results: mean over the 12 unseen test houses (`outputs/model_metrics_mean.csv`)

Model	Daily energy MAE (kWh/day)	Mean abs. total-energy error	Minute F1	Cycle recall	Cycle precision	MAE (W)	MAE when on (W)
Always-off	0.336	100%	0.00	0.00	–	**16.3**	463
Hour-of-week profile	0.384	71%	0.10	0.00	–	31.2	440
XGBoost window	0.249	**40%**	0.39	0.81	0.39	21.9	**313**
XGBoost window + cycle filter (final)	**0.242**	**40%**	**0.43**	**0.81**	**0.39**	20.5	318
Final, *without* time-of-day features	0.244	44%	0.42	0.80	0.38	21.1	322

Per test house (final model):

Test house	True kWh	Predicted kWh	Energy error	Daily MAE: model	Daily MAE: always-off	Minute F1	Cycle recall	Cycle precision
1	127	120	−5%	0.15	0.20	0.35	0.81	0.42
2	195	220	+13%	0.25	0.32	0.35	0.91	0.26
3	322	520	+62%	0.45	0.52	0.49	0.89	0.36
4	185	80	−57%	0.21	0.29	0.59	0.98	0.59
5	344	214	−38%	0.42	0.53	0.21	0.64	0.27
7	470	497	+6%	0.27	0.76	0.62	0.98	0.49
15	196	82	−58%	0.21	0.34	**0.81**	0.86	**0.88**
16	206	161	−22%	0.26	0.38	0.36	0.92	0.37
17	106	94	−11%	0.14	0.23	0.55	0.91	0.45
18	48	97	+103%	0.15	0.11	0.38	0.90	0.25
19	65	35	−46%	0.12	0.14	0.19	**0.17**	0.16
21	106	171	+62%	0.26	0.22	0.24	0.78	0.20

Pooled over all 12 houses, predicted washing-machine energy is within **−3%** of the truth. Over- and under-estimates cancel out across houses.

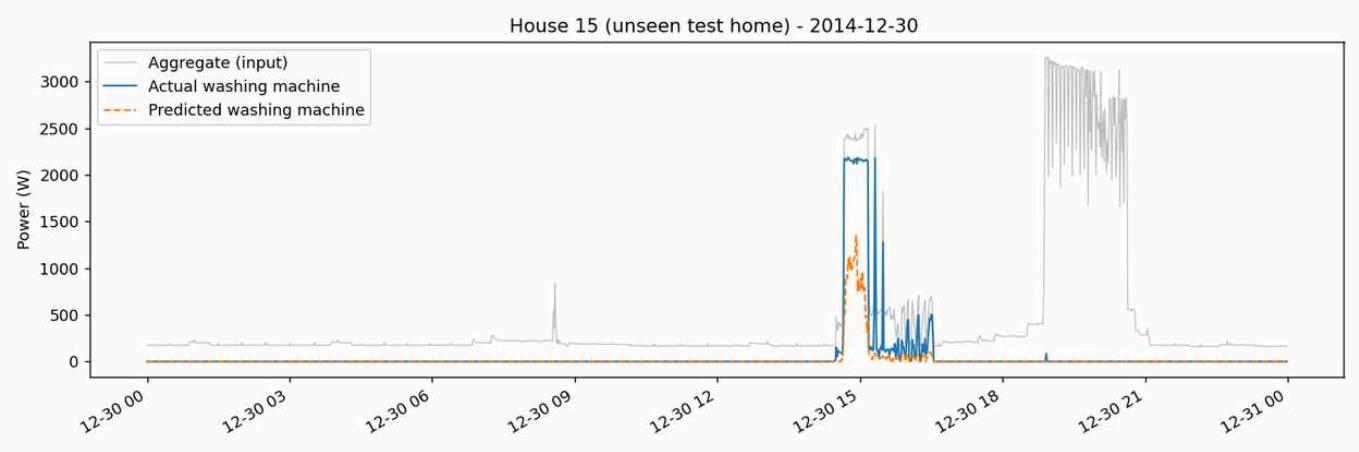
Interpretation

- **The model clearly beats both baselines on every metric a utility would use.** Daily energy error is 28% lower than always-off and better in 10 of 12 homes. It finds **81% of real washes** in homes it has never seen, and minute F1 is four times that of the usage profile.
- **Per-minute MAE is the one metric where always-off wins** (16 W vs 21 W). With the machine off 92–99% of the time, any false alarm costs more than predicting zero everywhere. This is a known weakness of MAE for rarely used appliances, and the reason MAE is not the headline metric.

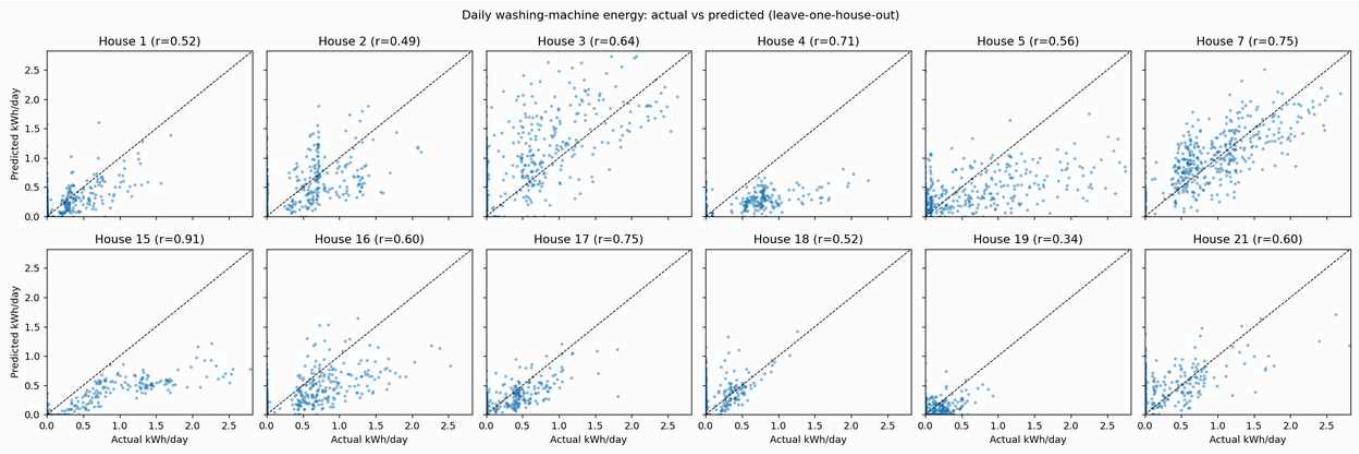
- **The main weakness is false alarms.** Only about 39% of predicted washes are real, because other heating loads look like the 2 kW plateau. Estimates for a *single* home can be off by ±50–100% in total energy. Results should be reported as aggregated or ranked insights, not exact per-home kWh.
- **Ablation:** removing the hour and weekend features barely changes anything (F1 0.43 → 0.42). The model recognises the *shape* of a wash in the aggregate rather than memorising when people wash, which is what lets it generalise to new homes. The most important features (`outputs/model_feature_importance.csv`) are level above the local baseline, the 5-minute minimum (a sustained load, not a spike) and the time spent above 1.5 kW in 15–61 minute windows.
- The cycle filter mainly removes false energy: mean signed total error drops from +16% to +0.6%, and minute F1 rises from 0.39 to 0.43.

Actual vs predicted on unseen homes

Good case, **House 15**, 31 Jan 2015 (`figures/p3_actual_vs_pred_h15.png`). The heating plateau and its timing are captured almost exactly. The low tail is partly missed, which is why House 15's energy is under-estimated.

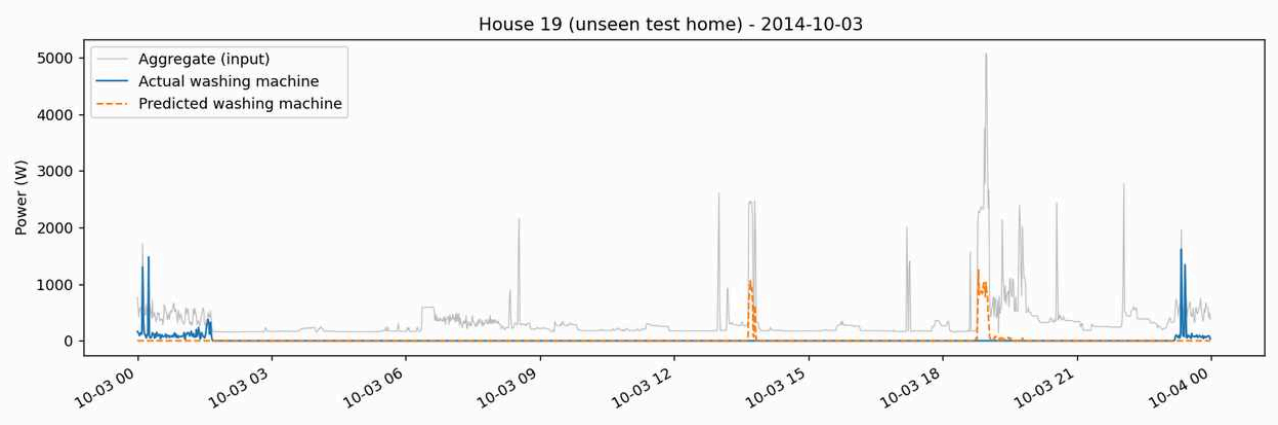


Daily energy for every test home (`figures/p3_daily_energy_scatter.png`). Points on the dashed line are perfect. The model separates wash days from non-wash days well (r = 0.5–0.86 in most homes) but compresses high-energy days toward the middle.

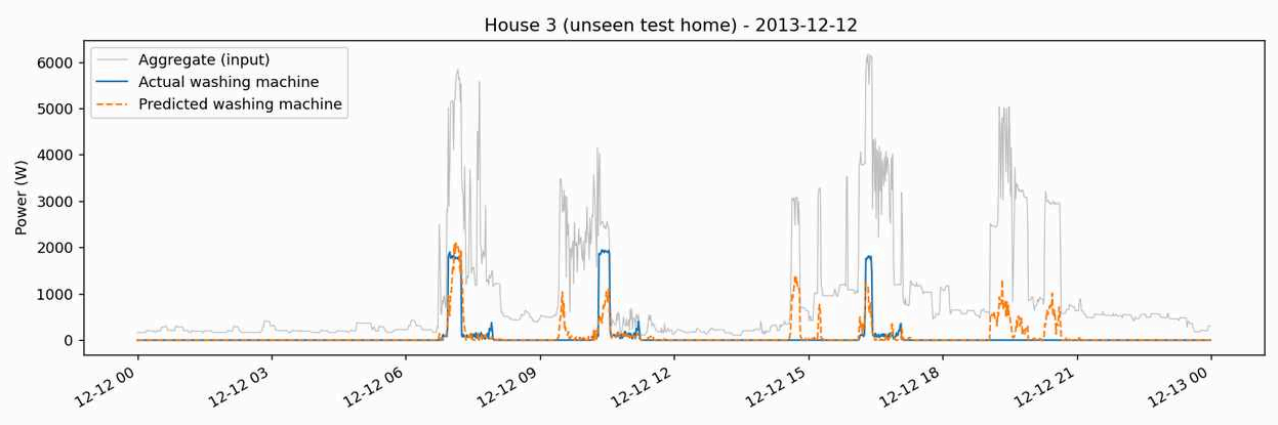


Failure cases

1. **A washing machine with an unusual signature, House 19** (`figures/p3_actual_vs_pred_h19.png`). This machine heats in short 1.3–1.6 kW bursts instead of one long 2 kW plateau, and mostly at night. None of the 11 training homes has that pattern, so the model misses 83% of its washes. On the same day it fires on two 2.4 kW plateaus from another appliance (13:40 and 18:45), likely a dishwasher or oven. *Lesson:* an unusual machine in a new home is the biggest risk. A small amount of labelled data or user feedback from that home ("did you wash today?") would help most.



2. **Busy homes with overlapping heating loads, House 3 (+62%) and House 21 (+62%)** (`figures/p3_actual_vs_pred_h3.png`). When the aggregate is full of 2–6 kW loads (electric shower, cooker, tumble dryer or dishwasher heating), several of them get labelled as washing. Washes that overlap another big load are also under-estimated, because the plateau is hidden inside a larger block. House 1 is a related case: its washer-dryer (not part of the target) produces true washing-like cycles that the model correctly sees but that count as false positives.



Research used

Paper	What I took from it
Murray, Stankovic & Stankovic (2017), *An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study*, Scientific Data 4:160122	Dataset description, IAM limitations, the Issues flag (which I then tested and decided not to use)
Zhang, Zhong, Wang, Goddard & Sutton (2018), *Sequence-to-point learning with neural networks for non-intrusive load monitoring*, AAAI	Predicting the midpoint of an aggregate window; evaluating on unseen REFIT houses
Kelly & Knottenbelt (2015), *Neural NILM: Deep neural networks applied to energy disaggregation*, ACM BuildSys	Per-appliance on-power threshold (~20 W for washing machines); training on windows that are rebalanced toward activations; reporting both energy and on/off metrics
Makonin & Popowich (2015), *Nonintrusive load monitoring (NILM) performance evaluation*, Energy Efficiency 8	Why accuracy and MAE mislead for sparse appliances, and why energy-based and state-based metrics should be reported side by side
D'Incecco, Squartini & Zhong (2019), *Transfer learning for non-intrusive load monitoring*, IEEE Trans. Smart Grid	Cross-house generalisation is the realistic test, and the motivation for the leave-one-house-out design

What I would do next

1. **Event-level confirmation:** a second-stage classifier on each predicted cycle as a whole (duration, energy, tail shape, time since last wash) to cut the 61% false-alarm rate. It would use the same LOHO validation.
2. **A Seq2Point CNN** with the same folds, to see whether learned features capture the low tail better than hand-made ones.
3. **Add the remaining REFIT houses** (6, 8, 9, 10, 13, 20) after manually checking their channel labels, so more machine types are seen in training.
4. **Probabilistic outputs**, such as an interval on monthly washing-machine kWh, so a utility can decide when an insight is confident enough to show a customer.